PROBLEM 1 (20 points) Consider the following type declarations:
TYPE
T1 : int
T2 : pointer to int
T3 : pointer to int
T4 : structure {a: int;}
T5 : structure {b: T3; a: pointer to T6;}
T6 : structure {a: T2; b: pointer to T5;}
T7 : array [2][3] of T5
T8 : array [2][3] of T6
1. (10 points) Which types are structurally equivalent? Show the step by step table.

	T1	T2	T3	T4	T5	T6	T7	T8
T1	T	T	T	T	T	T	T	T
T2	T	T	T	T	T	T	T	T
T3	T	T	T	T	T	T	T	T
T4	T	T	T	T	T	T	T	T
T5	T	T	T	T	T	T	T	T
T3	T	T	T	T	T	T	T	T
T7	T	T	T	T	T	T	T	T
T8	T	T	T	T	T	T	T	T

	T1	T2	T3	T4	T5	T6	T7	T8
T1	T	F	F	F	F	F	F	F
T2	F	T	T	T	T	T	T	T
T3	F	T	T	T	T	T	T	T
T4	F	T	T	T	T	T	T	T
T5	F	T	T	T	T	T	T	T
T3	F	T	T	T	T	T	T	T
T7	F	T	T	T	T	T	T	T
T8	F	T	T	T	T	T	T	T

	T1	T2	T3	T4	T5	T6	T7	T8
T1	T	F	F	F	F	F	F	F
T2	F	T	T	F	F	F	F	F
T3	F	T	T	T	T	T	T	T
T4	F	F	T	T	T	T	T	T
T5	F	F	T	T	T	T	T	T
T3	F	F	T	T	T	T	T	T
T7	F	F	T	T	T	T	T	T
T8	F	F	T	T	T	T	T	T

	T1	T2	T3	T4	T5	T6	T7	T8
T1	T	F	F	F	F	F	F	F
T2	F	T	T	F	F	F	F	F
T3	F	T	T	F	F	F	F	F
T4	F	F	F	T	T	T	T	T
T5	F	F	F	T	T	T	T	T
T3	F	F	F	T	T	T	T	T
T7	F	F	F	T	T	T	T	T
T8	F	F	F	T	T	T	T	T

	T1	T2	T3	T4	T5	T6	T7	T8
T1	T	F	F	F	F	F	F	F
T2	F	T	T	F	F	F	F	F
T3	F	T	T	F	F	F	F	F
T4	F	F	F	T	F	F	F	F
T5	F	F	F	F	T	T	T	T
T3	F	F	F	F	T	T	T	T
T7	F	F	F	F	T	T	T	T
T8	F	F	F	F	T	T	T	T

	T1	T2	T3	T4	T5	T6	T7	T8
T1	T	F	F	F	F	F	F	F
T2	F	T	T	F	F	F	F	F
T3	F	T	T	F	F	F	F	F
T4	F	F	F	T	F	F	F	F
T5	F	F	F	F	T	T	F	F
T6	F	F	F	F	T	T	T	T
T7	F	F	F	F	F	T	T	T
T8	F	F	F	F	F	T	T	T

	T1	T2	T3	T4	T5	T6	T7	T8
T1	T	F	F	F	F	F	F	F
T2	F	T	T	F	F	F	F	F
T3	F	T	T	F	F	F	F	F
T4	F	F	F	T	F	F	F	F
T5	F	F	F	F	T	T	F	F
T6	F	F	F	F	T	T	F	F
T7	F	F	F	F	F	F	T	T
T8	F	F	F	F	F	F	T	T

	T1	T2	T3	T4	T5	T6	T7	T8
T1	T	F	F	F	F	F	F	F
T2	F	T	T	F	F	F	F	F

T3	F	T	T	F	F	F	F	F
T4	F	F	F	T	F	F	F	F
T5	F	F	F	F	T	T	F	F
T6	F	F	F	F	T	T	F	F
T7	F	F	F	F	F	F	T	T
T8	F	F	F	F	F	F	T	T

	T1	T2	T3	T4	T5	T6	T7	T8
T1	T	F	F	F	F	F	F	F
T2	F	T	T	F	F	F	F	F
T3	F	T	T	F	F	F	F	F
T4	F	F	F	T	F	F	F	F
T5	F	F	F	F	T	T	F	F
T6	F	F	F	F	T	T	F	F
T7	F	F	F	F	F	F	T	T
T8	F	F	F	F	F	F	T	T

T2 and T3 are structurally equivalent because they are pointers to int.

T5 and T6 are structurally equivalent because they have same number and type of objects.

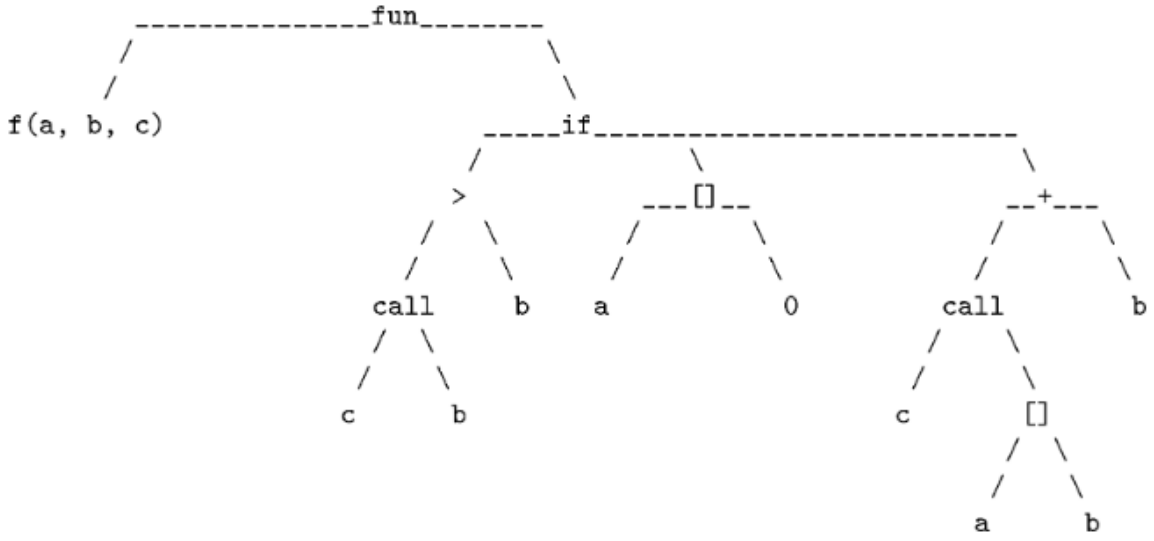
T7 and T8 are structurally equivalent because both arrays have the same dimensions and T5 and T6 are structurally equivalent.

Now Consider the following variable declarations in conjunction to the above type declarations:
VAR
x : T5;
y : T6;
z : T7;
w : T8;
u, v : pointer to T4;
p : T5;
2. (2.5 points each blank) Assume that assignments between variables are allowed if the types of the variables are equivalent. For each of the following, list all type equivalence schemes under which the expression is valid. Consider name equivalence, internal name equivalence, and structural equivalence for each case. Assume that if two variables are equivalent under name equivalence, they are also equivalent under internal name equivalence.

- x = y; structurally equivalent
- u = v; internal name equivalent, structurally equivalent
- w = z; structurally equivalent
- x = p; name equivalent, internal name equivalent, structurally equivalent

PROBLEM 2 (20 points, 5 points each)
fun f (a, b, c) = if c(b) > b then
a[0]
else
c(a[b]) + b

The abstract syntax tree of f is the following:



(Note: 'call' is similar to the 'apply', we had used in the lecture slides)
Fill in the blank, reducing all types to basic types and type constructors if possible:

- Type of a = array of ints
- Type of b = int
- Type of c = function that takes an int and returns an int
- Type that f returns (array of ints, int, (int->int)) -> int

PROBLEM 3 Consider the following code in C syntax (use static scoping):

```
# include <stdio.h>
int bar (int i)
{
i = 15;
return i + 5;
}
int foo (int e, int y)
{
int a;
int i;
a = e + y;
e = e + 2;
i = y;
y = i - 1;
return bar (e);
}
int main ()
{
int a = 0;
int e[3] = {1, 2, 3};
int x;
x = foo (a, e[a]);
printf ("%d -%d -%d -%d -%d\n", a, e[0], e[1], e[2], x);
return 0;
}
```

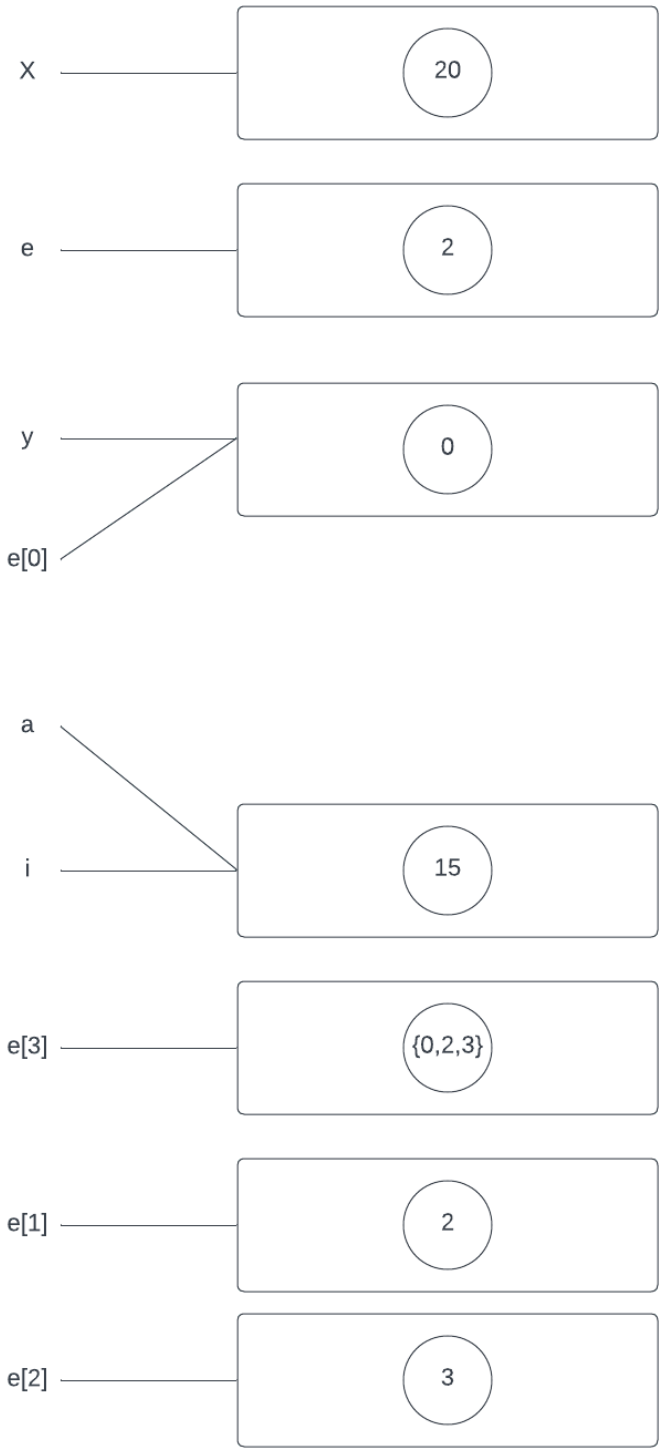
• (5 points) If parameters are passed by value, the output of this program is (Explain by showing the call arguments to each function)

0 -1 -2 -3 -20

Main	
a	0
e{3}	{1,2,3}
x	foo{0,a[0]=1}
Foo	
e	0
y	1
Bar	
e	2

• (5 points) If parameters are passed by reference, the output of this program is (explain with box and circle)

15 -0 -2 -3 -20



PROBLEM 4 (4 points each) Fully β-reduce the following expressions, show β reduction steps.

1. (λ x . x (λ x . λ y . y) (λ y . λ x . y)) (λ x . λ y . y)

(λ x . x (λ x . λ y . y) (λ y . λ x . y)) (λ x . λ y . y)

(((x [x->(λ x . λ y . y)]) (λ y . λ x . y)) (λ x . λ y . y)

((λ x . λ y . y) (λ y . λ x . y)) (λ x . λ y . y)

(λ y . y[x->(λ y . λ x . y)]) (λ x . λ y . y)

(λ y . y) (λ x . λ y . y)

(y[y->(λ y . λ x . y)])

λ y . λ x . y

2. (λa. λe. a e) (a e x)

(λa. a e x) (a e x)

(λe. a e [a-> a e x])

λe. a e x e not correct!

(λf. a e {f/e} [a-> a e x])

λf. a e x f

3. (λa. λi. i a) (alpha) (λa. a a)

(λa. λi. i a) (alpha) (λa. a a)

(λi. i a [a->(alpha)]) (λa. a a)

(λi. i alpha) (λa. a a)

(i alpha [i->(λa. a a)])

(λa. a a) alpha

(a a[a->(alpha)])
alpha alpha

4. (λ x . x λ y . λ z . z y x) (y z)

(λ x . x λ y . λ z . z y x) (y z)

(x λ y . λ z . z y x [x->y z])

y z λ y . λ z . z y y z not correct!

(x λ w . λ u . z y x {w/y}{u/z} [x->(y z)])

yz λw. λu. wuyz

5. ((λe. e e) (λa. a)) (ae)

((λe. e e) (λa. a)) (a e)

(ee[e->(λa. a)]) (a e)

((λa. a) (λa. a)) (a e)

(a[a->(λa. a)]) (a e)

(λa. a) (a e)

(a[a->(ae)]) (a e)

ae

Problem 5 (2.5 points each) Find the correct binders to the variables in the expressions and determine whether each λ-expression is open or closed. A closed expression is an expression without any free variables.

(λz. y z x) z λx. λz. x y z (λz. y z x z)
(λ**z**. y **z** x) **z** λx. λ**z**. x y **z** (λ**z**. y **z** x **z**)

(λz. y z x) z λx. λz. x y z (λz. y z x z) is an open λ-expression because y is a free variable.

(λx. (λz. x y) z)
(λ**x**. (λz. **x** y) z)

(λx. (λz. x y) z) is an open λ-expression because y is a free variable.

λx. λy. λx. x (λz. x y z)

λ**x**. λy. λ**x**. **x** (λ*z*. **x** y *z*)

λx. λy. λx. x (λz. x y z) is a closed λ-expression because none of the variables is a free variable

λx. λx. z (λx. x y z)

λ**x**. λ**x**. z (λ**x**. **x** y z)

λx. λx. z (λx. x y z) is an open λ-expression because y and z are free variables.